

3.1 UNE VUE = LE CHEF DE SALLE

Analogie restaurant :

- Le client (navigateur) commande un plat (URL)
- Le chef de salle (vue) note la commande, va en cuisine (base de données), récupère les données, et apporte l'assiette présentée (template HTML)

La vue est la fonction Python qui reçoit une requête HTTP et renvoie une réponse HTML, JSON, ou PDF.

STRUCTURE TYPE D'UNE VUE SIMPLE :

```
from django.shortcuts import render
from .models import Produit

def liste_produits(request):
    produits = Produit.objects.all().order_by('-stock') # cuisine
    contexte = {'produits': produits, 'titre': 'Mes Produits'}
    return render(request, 'boutique/liste_produits.html', contexte)
```

DÉCOMPOSITION :

request → Ce que le client envoie (méthode GET/POST, données)
Produit.objects → La cuisine (accès base de données)
contexte → Le plateau (données envoyées au template)
render(...) → Le service à table

3.2 LES QUATRE ACTIONS FONDAMENTALES (CRUD)

CRUD = Create / Read / Update / Delete

Toute gestion de données se résume à ces 4 actions.

ACTION	CODE DJANGO CORRESPONDANT
CREATE	Produit.objects.create(nom="Riz", prix_vente=800)
READ	Produit.objects.get(id=1) Produit.objects.filter(stock__gt=0)
UPDATE	p = Produit.objects.get(id=1); p.stock = 30; p.save()
DELETE	Produit.objects.get(id=1).delete()

3.3 PERSONNALISATION CONTEXTUELLE – EXEMPLE MICROFINANCE

CONTEXTE ORIGINAL (vente de produits) :

```
def tableau_de_bord(request):
    total_ventes = Vente.objects.aggregate(Sum('montant'))['montant__sum'] or 0
    return render(request, 'dashboard.html', {'total_ventes': total_ventes})
```

ADAPTATION MICROFINANCE (prêts à la place des ventes) :

```
from django.db.models import Sum, Count
from .models import Pret, Remboursement

def tableau_de_bord(request):
    encours_total = Pret.objects.filter(statut='actif').aggregate(
        Sum('montant'))['montant__sum'] or 0
    taux_recouvrement = _calculer_taux_recouvrement()
    prets_en_retard = Pret.objects.filter(statut='retard').count()

    return render(request, 'dashboard.html', {
        'encours_total': encours_total,
        'taux_recouvrement': taux_recouvrement,
        'prets_en_retard': prets_en_retard,
    })
```

```
def _calculer_taux_recouvrement():
    total_du = Pret.objects.aggregate(Sum('montant'))['montant__sum'] or 1
    total_rembourse = Remboursement.objects.aggregate(Sum('montant'))['montant__sum'] or 0
    return round((total_rembourse / total_du) * 100, 1)
```

LEÇON : Seules les lignes qui touchent aux modèles changent.
 La structure render/contexte/template reste identique.

3.4 TRAITER UN FORMULAIRE – LE MOTIF GET/POST

Ce motif revient dans 80% des vues qui modifient des données.
 Mémorisez-le comme une formule chimique.

```
def creer_produit(request):
    if request.method == 'POST':
        # L'utilisateur a soumis le formulaire → on traite
        form = ProduitForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, "Produit créé avec succès.")
            return redirect('liste_produits')
    else:
        # L'utilisateur arrive pour la première fois → formulaire vide
        form = ProduitForm()

    return render(request, 'creer_produit.html', {'form': form})
```

RÈGLE : GET = afficher. POST = traiter et sauvegarder.
 Cette règle ne change jamais, quel que soit le contexte.

3.5 FILTRES AVANCÉS – ADAPTER LES REQUÊTES À VOTRE CONTEXTE

BESOIN	CODE DJANGO
Produits dont prix > 1000 FCFA	.filter(prix_vente__gt=1000)
Ventes du mois de janvier	.filter(date__month=1)
Clients dont nom contient "Ba"	.filter(nom__icontains="ba")
10 produits les plus vendus	.order_by('-quantite_vendue')[:10]
Produits sans stock	.filter(stock=0)
Ventes entre deux dates	.filter(date__range=[d1, d2])

Ces filtres sont votre boîte à outils principale pour adapter
 n'importe quelle vue à n'importe quel contexte.

RÉSUMÉ DU CHAPITRE 3

- ✓ La vue reçoit la requête, interroge la base, renvoie un template
 - ✓ CRUD = les 4 actions de base de toute gestion de données
 - ✓ Adapter une vue = changer les modèles interrogés, pas la structure
 - ✓ GET = afficher | POST = traiter (règle universelle)
 - ✓ Les filtres Django couvrent 95% des besoins de recherche
- SUITE : Chapitre 4 – Les templates HTML et le contexte visuel